

DSAA3073: Theories in Data Science

Week 1: Mathematical Preliminaries

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-29

Explorative projection

Introduction: Why Mathematics for Machine Learning?

Machine learning seems simple: collect data, fit a model, make predictions. But this intuitive process hides subtle mathematical questions that determine whether learning is possible:

Opening challenge: You train a classifier on 1000 samples and achieve 95% training accuracy. You deploy it to production.

Questions:

- What can you guarantee about test accuracy?
- With what probability will test accuracy exceed 90%?
- How many more samples would guarantee 99% confidence?
- Does the answer depend on model complexity?

This week's goal: Build the mathematical foundation to answer these questions rigorously.

This Week's Learning Objectives

After completing this material, you should be able to:

1. Concentration Inequalities:

- Explain why Markov and Chebyshev bounds are too loose for learning
- Apply Hoeffding's inequality to compute sample complexity
- Understand exponential vs polynomial tail decay

2. Norms and Geometry:

- Explain why different norms give different solutions
- Choose appropriate norms for ML problems (L^1 for sparsity, L^2 for stability)
- Apply Cauchy-Schwarz inequality

3. Convexity:

- Identify when optimization problems are convex
- Explain why neural networks are harder to train than logistic regression
- Understand convergence guarantees ($O(1/t)$ vs $O(\exp(-t))$)

4. Complexity Theory:

- Distinguish tractable (polynomial) from intractable (exponential) problems
- Explain why approximation algorithms are needed for NP-hard problems
- Understand the computational-statistical gap

The Learning Theory Pipeline

This week establishes tools that Week 2 combines into PAC learning:

$$\text{Probability} \longrightarrow \text{Geometry} \longrightarrow \text{Optimization} \longrightarrow \text{Complexity} \quad (1)$$

Concretely:

1. **Probability:** Hoeffding bounds sample mean deviations
2. **Geometry:** Norms measure hypothesis space structure
3. **Optimization:** Convexity guarantees finding good solutions
4. **Complexity:** Polynomial time ensures tractability

The connection: These four pillars enable the PAC learning framework in Week 2.

Concentration Inequalities: From Loose to Tight

🧐 Student Question

From the class: “I want to estimate the probability that my model’s error exceeds 10%. I know the average error is 5%. Can’t I just use Markov’s inequality: $P(\text{error} \geq 10\%) \leq 5\%/10\% = 50\%$? That seems like enough.”

This seems simple and general—Markov only needs the mean, no other assumptions!

The Natural Starting Point: Markov’s Inequality

When you first learn about bounding tail probabilities, Markov’s inequality seems perfect: it’s simple, general, and requires minimal information.

Markov’s inequality: For non-negative random variable X and $t > 0$,

$$\text{Prob}(X \geq t) \leq \frac{\text{Exp}[X]}{t} \quad (2)$$

Why it’s appealing:

- Only needs the mean (no variance, no boundedness assumptions)
- Works for any non-negative distribution
- Easy to apply: just divide mean by threshold

Example application (learning):

- Average error on validation set: $\text{Exp}[\text{error}] = 5\%$
- Want to bound: $\text{Prob}(\text{error} \geq 10\%)$
- Markov gives: $\text{Prob}(\text{error} \geq 10\%) \leq \frac{0.05}{0.10} = 0.5$

Interpretation: With 50% confidence, error is below 10%. Seems reasonable?

Where Markov Breaks: The Sample Complexity Disaster

Let’s apply Markov to a concrete learning problem.

Scenario: Estimate true accuracy from a validation set

- True accuracy: $p = 0.90$ (90% correct)
- Validation set size: m samples
- Empirical accuracy: $\hat{p} = \frac{1}{m} \sum_{i=1}^m X_i$ where $X_i \in \{0, 1\}$
- Question: How large must m be to guarantee $|\hat{p} - p| \leq 0.05$ with 95% confidence?

Using Markov naively:

We want $\text{Prob}(|\hat{p} - 0.90| \geq 0.05) \leq 0.05$.

Problem: Markov requires non-negative variables. Let $Y = |\hat{p} - p|$ (absolute deviation).

Markov says: $\text{Prob}(Y \geq 0.05) \leq \frac{\text{Exp}[Y]}{0.05}$

To make this ≤ 0.05 , we need: $\text{Exp}[Y] \leq 0.0025$

But computing $\text{Exp}[\hat{p} - p]$ is hard without more structure!

Better attempt: Bound $\text{Prob}(\hat{p} \geq 0.95)$ (one-sided)

Markov: $\text{Prob}(\hat{p} \geq 0.95) \leq \frac{\text{Exp}[\hat{p}]}{0.95} = \frac{0.90}{0.95} \approx 0.95$

This bound is terrible! It says “95% probability” when we want 5%.

⚠ Crisis

The Markov Catastrophe: Bounds Too Loose for Learning

Problem: Using Markov for learning guarantees

What we get:

- Want: $\text{Prob}(\text{error} \geq 0.10) \leq 0.05$ (5% failure rate)
- Markov gives: $\text{Prob}(\text{error} \geq 0.10) \leq 0.50$ (50% failure rate)
- Gap: 10× worse than needed!

Impact on sample complexity:

To achieve $\text{Prob}(\text{error} \geq \varepsilon) \leq \delta$ using Markov, need:

$$\frac{\text{Exp}[\text{error}]}{\varepsilon} \leq \delta \Rightarrow \text{Exp}[\text{error}] \leq \varepsilon \cdot \delta \quad (3)$$

For $\varepsilon = 0.10$, $\delta = 0.05$: need $\text{Exp}[\text{error}] \leq 0.005$ (0.5% average error!)

This translates to: **need 40,000 samples** when reality needs only 400

The fundamental flaw: Polynomial decay ($\frac{1}{t}$) is too slow for practical learning

First Improvement: Chebyshev Uses Variance

The key insight: variance captures spread around the mean.

💡 Aha Moment

The Resolution: Use Variance

Markov only uses first moment (mean). What if we use the second moment (variance)?

Chebyshev’s trick: Apply Markov to $(X - \mu)^2$ (always non-negative!)

Result: $\text{Prob}(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$

Key improvement: Decay is now $\frac{1}{k^2}$ (polynomial) instead of $\frac{1}{k}$

Theorem: Chebyshev's Inequality

For random variable X with finite variance $\sigma^2 = \text{Var}(X)$ and any $k > 0$:

$$\text{Prob}(|X - \text{Exp}[X]| \geq k) \leq \frac{\text{Var}(X)}{k^2} \quad (4)$$

In words: Probability of deviation by k is bounded by $\frac{\sigma^2}{k^2}$

Proof sketch:

1. $(X - \mu)^2$ is non-negative
2. Apply Markov with threshold k^2 :

$$\text{Prob}((X - \mu)^2 \geq k^2) \leq \frac{\text{Exp}[(X - \mu)^2]}{k^2} = \frac{\sigma^2}{k^2} \quad (5)$$

3. Note: $(X - \mu)^2 \geq k^2$ iff $|X - \mu| \geq k$

Why it's better: For large deviations ($k \gg \sigma$), bound $\frac{\sigma^2}{k^2}$ is much smaller than $\frac{\mu}{k}$.

Applying Chebyshev to Learning

Back to our problem: Estimate $p = 0.90$ from m samples

Empirical estimate: $\hat{p} = \frac{1}{m} \sum_{i=1}^m X_i$ where $X_i \in \{0, 1\}$

Key facts:

- $\text{Exp}[\hat{p}] = p = 0.90$
- $\text{Var}(\hat{p}) = \frac{p(1-p)}{m} = \frac{0.09}{m}$ (variance decreases with m !)

Apply Chebyshev:

$$\text{Prob}(|\hat{p} - 0.90| \geq 0.05) \leq \frac{\frac{0.09}{m}}{0.0025} = \frac{36}{m} \quad (6)$$

Sample complexity: To make this ≤ 0.05 : need $\frac{36}{m} \leq 0.05$, so $m \geq 720$ samples

 Comparison**Sample complexity comparison:****Markov (loose):**

- Cannot easily bound two-sided deviation
- Rough estimate: 40,000 samples needed

Chebyshev (better):

- Uses variance: $\text{Var}(\hat{p}) = \frac{p(1-p)}{m} \approx \frac{0.09}{m}$
- Sample bound: $m \geq 720$ samples
- **50× improvement!**

But can we do even better?

The Key Question: Can We Do Better?

Chebyshev gives $m \geq 720$, but this still seems large. What's limiting it?

Analysis of Chebyshev's bound:

- Decay rate: $\frac{1}{k^2}$ (polynomial)
- For $k = 0.05$: $\frac{1}{(0.05)^2} = 400$
- Multiplied by variance $\sigma^2 = \frac{0.09}{m}$

The bottleneck: Polynomial decay means we need large m to overcome the $\frac{1}{k^2}$ term.

Intuition: Can we get exponential decay? Something like $e^{-m\varepsilon^2}$?

That would make the bound shrink much faster!

The Breakthrough: Hoeffding's Exponential Bound

💡 Aha Moment

The Resolution: Exponential Concentration

Key insight: If random variables are bounded and independent, we can get exponential tail decay!

Instead of $O(\frac{1}{k^2})$ (polynomial), get $O(e^{-m\varepsilon^2})$ (exponential)

The technique: Exponential moment method (Chernoff bound)

Theorem: Hoeffding's Inequality

Let $X_1, \dots, X_m$ be independent random variables with $X_i \in [a_i, b_i]$.

Define sample mean: $\bar{X} = \frac{1}{m} \sum_{i=1}^m X_i$

Then for any $\varepsilon > 0$:

$$\text{Prob}(|\bar{X} - \text{Exp}[\bar{X}]| \geq \varepsilon) \leq 2 \exp\left(\frac{-2m^2\varepsilon^2}{\sum_{i=1}^m (b_i - a_i)^2}\right) \quad (7)$$

For i.i.d. variables in $[a, b]$:

$$\text{Prob}(|\bar{X} - \text{Exp}[X]| \geq \varepsilon) \leq 2 \exp\left(\frac{-2m\varepsilon^2}{(b - a)^2}\right) \quad (8)$$

Key properties:

1. **Exponential decay:** Probability decreases as $e^{-m\varepsilon^2}$ (not $\frac{1}{m}\varepsilon^2$!)
2. **Distribution-free:** Only needs boundedness, not specific distribution
3. **Optimal rate:** Cannot do better than $e^{-m\varepsilon^2}$ in general

Proof sketch (exponential moment method):

1. Transform tail probability:

$$\text{Prob}(\bar{X} - \mu \geq \varepsilon) = \text{Prob}(e^{t(\bar{X}-\mu)} \geq e^{t\varepsilon}) \quad (9)$$

2. Apply Markov to exponential:

$$\text{Prob}(e^{t(\bar{X}-\mu)} \geq e^{t\varepsilon}) \leq \frac{\text{Exp}[e^{t(\bar{X}-\mu)}]}{e^{t\varepsilon}} \quad (10)$$

3. Use independence:

$$\text{Exp}[e^{t \sum_i (X_i - \mu_i)}] = \text{product}_{i=1}^m \text{Exp}[e^{t(X_i - \mu_i)}] \quad (11)$$

4. Hoeffding's lemma: For $X \in [a, b]$ with $\text{Exp}[X] = 0$:

$$\text{Exp}[e^{tX}] \leq e^{t^2 \frac{(b-a)^2}{8}} \quad (12)$$

5. Optimize over t and apply union bound for two-sided inequality Key Takeaway

Why exponential moment method works:

1. Exponential function e^{tX} amplifies large deviations
2. Independence allows factorization of expectations
3. Boundedness gives tight control on moment generating function
4. Optimization over t balances Markov looseness with exponential growth

Applying Hoeffding to Learning

Back to our running example: Estimate $p = 0.90$ from m samples

Variables: $X_i \in \{0, 1\}$ (bounded in $[0, 1]$)

Apply Hoeffding:

$$\text{Prob}(|\hat{p} - 0.90| \geq 0.05) \leq 2 \exp(-2m \cdot 0.0025) = 2 \exp(-0.005m) \quad (13)$$

Sample complexity: To make this ≤ 0.05 :

$$2 \exp(-0.005m) \leq 0.05 \quad (14)$$

$$\exp(-0.005m) \leq 0.025 \quad (15)$$

$$-0.005m \leq \log(0.025) \approx -3.69 \quad (16)$$

$$m \geq \frac{738}{0.005} \approx 148 \quad (17)$$

So we need approximately 150 samples!

Comparison

Final sample complexity comparison:

Method	Bound Type	Samples Needed	Notes
Markov	$\frac{1}{t}$	40,000	Too loose, rarely used
Chebyshev	$\frac{1}{k^2}$	720	Uses variance, practical
Hoeffding	$e^{-m\varepsilon^2}$	148	Exponential, optimal!

The improvement: 40,000 $\rightarrow$ 720 $\rightarrow$ 148 samples

Key insight: Exponential decay makes learning tractable!

Without Hoeffding's exponential bound, we'd need 270 $\times$ more samples—making machine learning impractical.

The Concentration Hierarchy

The progression Markov $\rightarrow$ Chebyshev $\rightarrow$ Hoeffding reveals a pattern:

Inequality	Assumptions	Information Used	Tail Decay
Markov	Non-negative	Mean only	$O(\frac{1}{t})$
Chebyshev	Finite variance	Mean + variance	$O(\frac{1}{k^2})$
Hoeffding	Bounded + independent	Bounds + independence	$O(e^{-m\varepsilon^2})$

Pattern: More assumptions $\rightarrow$ tighter bounds

Trade-off: Generality vs tightness

- Markov: most general, loosest
- Hoeffding: more restrictive, exponentially tighter

For learning theory: Hoeffding's assumptions (bounded losses, independent samples) are natural, so we get exponential concentration!

Numerical Verification

Let's verify with actual numbers for $\varepsilon = 0.05$, $\delta = 0.05$, $\sigma^2 = 0.09$:

Markov (one-sided, rough): $\text{Prob}(X \geq \mu + \varepsilon) \leq \frac{\mu}{\mu + \varepsilon} \approx 0.95$ (useless!)

Chebyshev:

$$\text{Prob}(|\bar{X} - \mu| \geq 0.05) \leq \frac{\frac{0.09}{m}}{0.0025} = \frac{36}{m} \quad (18)$$

Need: $m \geq 720$

Hoeffding:

$$\text{Prob}(|\bar{X} - \mu| \geq 0.05) \leq 2 \exp(-2m \cdot 0.0025) = 2 \exp(-0.005m) \quad (19)$$

Need: $m \geq 148$

With $m = 1000$ samples:

- Chebyshev: $\frac{36}{1000} = 0.036$ (3.6% probability)
- Hoeffding: $2 \exp(-5) \approx 0.0135$ (1.35% probability)
- Both satisfy $\delta = 0.05$, but Hoeffding is much tighter!

With $m = 10,000$ samples:

- Chebyshev: $\frac{36}{10000} = 0.0036$ (0.36%)
- Hoeffding: $2 \exp(-50) \approx 10^{-21}$ (essentially zero!)
- Hoeffding's exponential decay dominates for large m

Looking Ahead

We've shown that Hoeffding gives exponential concentration for the sample mean of a single random variable.

But in learning, we don't just estimate one quantity—we choose a hypothesis from a class $\mathcal{H}$ based on training data.

Key question for Week 2: How do we extend Hoeffding to handle selection from a hypothesis class?

Preview: This requires **uniform convergence**—bounding deviations simultaneously for all $h \in \mathcal{H}$. The union bound plus Hoeffding gives PAC sample complexity!

Proof of Hoeffding's Lemma

For completeness, here's the key technical lemma:

Theorem: Hoeffding's Lemma

Let X be a random variable with $X \in [a, b]$ and $\text{Exp}[X] = 0$.

Then for any $t > 0$:

$$\text{Exp}[e^{tX}] \leq e^{t^2 \frac{(b-a)^2}{8}} \quad (20)$$

Proof:

1. **Convexity trick:** e^{tx} is convex, so for $x \in [a, b]$:

$$e^{tx} \leq \frac{b-x}{b-a} e^{ta} + \frac{x-a}{b-a} e^{tb} \quad (21)$$

2. **Take expectations:**

$$\text{Exp}[e^{tX}] \leq \frac{b - \text{Exp}[X]}{b-a} e^{ta} + \frac{\text{Exp}[X] - a}{b-a} e^{tb} \quad (22)$$

Since $\text{Exp}[X] = 0$:

$$\text{Exp}[e^{tX}] \leq \frac{b}{b-a}e^{ta} + \frac{-a}{b-a}e^{tb} \quad (23)$$

3. **Define:** $p = -\frac{a}{b-a}$, so $1-p = \frac{b}{b-a}$

$$\text{Exp}[e^{tX}] \leq (1-p)e^{ta} + pe^{tb} = e^{\log[(1-p)e^{ta} + pe^{tb}]} \quad (24)$$

4. **Taylor expansion and optimization** (technical calculation):

$$\log[(1-p)e^{ta} + pe^{tb}] \leq \frac{t^2(b-a)^2}{8} \quad (25)$$

Therefore: $\text{Exp}[e^{tX}] \leq e^{t^2 \frac{(b-a)^2}{8}}$ ■

Key Takeaway

Why Hoeffding's lemma works:

- Convexity of exponential allows bounding by linear combination of endpoints
- Boundedness in $[a, b]$ controls the exponential moment
- The constant $\frac{1}{8}$ is optimal (cannot be improved in general)

Norms and Geometry: Why L² Isn't Always the Answer

🧐 Student Question

From the class: “A norm just measures the ‘size’ of a vector. The Euclidean norm (L²) is the natural one from geometry—it’s just the straight-line distance. Can’t we use it everywhere?”

This seems obvious—L² is how we measure distance in the physical world!

The Natural Approach: Always Use Euclidean (L²) Norm

The Euclidean norm feels natural because it matches our geometric intuition:

Definition: $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$

Why it seems universal:

- Measures straight-line distance (Pythagorean theorem)
- Rotation-invariant (all directions treated equally)
- Has nice inner product structure: $\|x\|_2^2 = \text{inner}(x, x)$
- Used in physics, engineering, everyday geometry

Applications in ML:

- Ridge regression: minimize $\|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2$
- Neural networks: minimize $\|y - f(x; \theta)\|_2^2$
- K-means: minimize $\sum_i \|x_i - \mu_{c_i}\|_2^2$

So why consider other norms?

Where L² Breaks: Sparse vs Dense Solutions

Let’s see what happens with different norms in a concrete optimization problem.

Problem: Minimize $f(x) + \text{regularization penalty}$

Setup:

- Loss function: $f(x) = (x_1 - 100)^2 + (x_2 - 1)^2 + (x_3 - 1)^2 + \dots + (x_{100} - 1)^2$
- Constraint: $\|x\| \leq 100$ (regularization)

Intuition: First coordinate “wants” to be 100, others want to be 1.

With L² norm constraint: $\|x\|_2 \leq 100$

Optimal solution: Distribute budget across all coordinates

- $x^* \approx (99.5, 0.995, 0.995, \dots, 0.995)$ (all coordinates non-zero)
- Result: **Dense solution** (100 non-zero entries)

With L^1 norm constraint: $\|x\|_1 \leq 100$

Optimal solution: Concentrate budget on important coordinates

- $x^* = (100, 0, 0, \dots, 0)$ (only first coordinate non-zero)
- Result: **Sparse solution** (1 non-zero entry!)

The difference: Same constraint budget (100), completely different solutions!

⚠ Crisis

The Norm Choice Crisis: Different Solutions from Different Norms

Problem: Optimization with norm constraint

What happens:

- L^2 norm: Encourages dense solutions (all coordinates contribute)
- L^1 norm: Encourages sparse solutions (few coordinates non-zero)
- L^∞ norm: Bounds maximum coordinate (different structure)

Concrete example: Problem: Fit model with 100 features, constraint budget = 100

L^2 regularization (Ridge):

- Solution: All 100 features have small weights
- Example: $\theta = (0.5, 0.3, 0.2, 0.1, 0.05, \dots)$ (all non-zero)
- Interpretation: All features matter a little

L^1 regularization (Lasso):

- Solution: Only 10 features have non-zero weights
- Example: $\theta = (2, 0, 0, 3, 0, 0, 1, 0, \dots)$ (90% zeros!)
- Interpretation: Feature selection—only important features

The gap: 100 features vs 10 features (10× sparsity!)

Impact: Norm choice is an inductive bias, not just a technicality

Understanding Why: Geometry of Norm Balls

The reason lies in the geometry of different norm balls.

Unit balls: $\{x : \|x\| \leq 1\}$

Comparison

Geometry of ℓ_p norms in $\mathbb{R}^2$:

L¹ norm: $\|x\|_1 = |x_1| + |x_2|$

- Unit ball: Diamond shape
- Corners at axes: $(\pm 1, 0)$ and $(0, \pm 1)$
- Property: Optimal points tend to lie on axes (sparsity!)

L² norm: $\|x\|_2 = \sqrt{x_1^2 + x_2^2}$

- Unit ball: Circle
- No special directions—all treated equally
- Property: Smooth boundary, no sparsity preference

L[∞] norm: $\|x\|_\infty = \max(|x_1|, |x_2|)$

- Unit ball: Square
- Corners at diagonals: $(\pm 1, \pm 1)$
- Property: Bounds maximum coordinate

Why sparsity happens with L¹: When a level set of the objective function (ellipse for quadratic) touches the L¹ ball, it typically hits a corner—which lies on an axis (sparse!)

For L² ball (circle), contact point is smooth—no preference for axes

The Key Insight: Norm Choice Encodes Prior Knowledge

Aha Moment

The Resolution: Norm Choice is Inductive Bias

Different norms encode different assumptions about solutions:

- **L¹:** “Solution should be sparse” (few non-zero features)
- **L²:** “Solution should have small, distributed weights” (all features contribute)
- **L[∞]:** “No single feature should dominate” (bounded max)

Norm choice determines solution structure, not just size!

Definition: Norm

A **norm** on vector space V is a function $\|\cdot\| : V \rightarrow \mathbb{R}_+$ satisfying:

1. **Positivity:** $\|x\| \geq 0$ with equality iff $x = 0$
2. **Homogeneity:** $\|\alpha x\| = |\alpha| \|x\|$ for scalars α
3. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$

Intuition: A norm assigns “size” to vectors, generalizing length

Definition: ℓ_p Norms

For $x \in \mathbb{R}^n$ and $p \geq 1$:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}} \quad (26)$$

Special cases:

- $\|x\|_1 = \sum_{i=1}^n |x_i|$ (sum of absolute values)
- $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$ (Euclidean distance)
- $\|x\|_\infty = \max_i |x_i|$ (maximum absolute value)

Applications in Machine Learning

Comparison

Norm choice in regularization:

Method	Penalty	Solution Type	Use Case
Ridge	$\lambda \ \theta\ _2^2$	Dense	Numerical stability
Lasso	$\lambda \ \theta\ _1$	Sparse	Feature selection
Elastic Net	$\lambda_1 \ \theta\ _1 + \lambda_2 \ \theta\ _2^2$	Sparse + stable	High-dimensional data
Max-norm	$\lambda \ \theta\ _\infty$	Bounded max	Robust to outliers

Example (gene expression data):

- 20,000 genes (features), 100 patients (samples)
- L^2 (Ridge): Uses all 20,000 genes with small weights (hard to interpret)
- L^1 (Lasso): Selects 50 genes with non-zero weights (interpretable!)
- **Sparsity: 20,000 $\rightarrow$ 50 features (400 $\times$ reduction!)**

Example: Ridge vs Lasso Regression**Ridge regression:**

$$\min_{\theta} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2 \quad (27)$$

Solution: $(X^T X + \lambda I)^{-1} X^T y$ (closed form!)

- All coefficients shrunk toward zero
- No coefficients exactly zero
- Good for prediction, hard to interpret

Lasso regression:

$$\min_{\theta} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1 \quad (28)$$

Solution: No closed form, use iterative methods

- Many coefficients exactly zero
- Automatic feature selection
- Interpretable models

Real example (Boston housing):

- 13 features, 506 samples
- Ridge: All 13 features non-zero
- Lasso ($\lambda=0.1$): Only 8 features non-zero
- **Automatic feature selection via sparsity!**

Why Norm Equivalence Doesn't Save Us

You might think: “In finite dimensions, all norms are equivalent, so they’re basically the same!”

Theorem: Norm Equivalence

In $\mathbb{R}^n$, for any norms $\|\cdot\|_a$ and $\|\cdot\|_b$, there exist $c, C > 0$ such that:

$$c \|x\|_a \leq \|x\|_b \leq C \|x\|_a \quad \text{for all } x \quad (29)$$

Why this doesn't mean norms are interchangeable:

1. Constants can be large

- In $\mathbb{R}^{1000}$: $\|x\|_2 \leq \|x\|_1 \leq \sqrt{1000} \|x\|_2$
- $\sqrt{1000} \approx 31.6$ gap makes bounds very loose!

2. Optimization favors different solutions

- Equivalent topologically (same convergence)
- Not equivalent for optimization (different optima!)

3. Inductive bias matters

- L^1 sparsity is a feature, not a bug
- We choose norms to encode prior knowledge

Key Takeaway

Norm equivalence means:

- Convergence in one norm $\rightarrow$ convergence in all norms
- All norms induce same topology (open sets)

Norm equivalence does NOT mean:

- Optimization gives same solutions
- Regularization has same effect
- Geometric properties are identical

Bottom line: Norm choice encodes inductive bias about solution structure

Inner Products and Angles

Some norms come from inner products, which add geometric structure:

Definition: Inner Product

An **inner product** $\text{inner}(\cdot, \cdot)$ satisfies:

1. Linearity: $\text{inner}(\alpha x + \beta y, z) = \alpha \text{inner}(x, z) + \beta \text{inner}(y, z)$
2. Symmetry: $\text{inner}(x, y) = \text{inner}(y, x)$
3. Positive-definiteness: $\text{inner}(x, x) \geq 0$ with equality iff $x = 0$

Induced norm: $\|x\| = \sqrt{\text{inner}(x, x)}$

Key property: L^2 norm comes from standard inner product $\text{inner}(x, y) = \sum_i x_i y_i$

But: L^1 and L^∞ norms do NOT come from inner products!

- Can't define angles with L^1 norm
- Can't project orthogonally with L^∞ norm

Impact on ML:

- PCA requires inner product (uses L^2)
- Kernel methods require inner product structure
- Some algorithms need L^2 for geometric operations

Cauchy-Schwarz: The Fundamental Bound

For norms induced by inner products, we have:

Theorem: Cauchy-Schwarz Inequality

For vectors x, y in an inner product space:

$$|\text{inner}(x, y)| \leq \|x\| \|y\| \quad (30)$$

with equality iff x and y are linearly dependent

Proof sketch: Consider $f(t) = \|x - ty\|^2 \geq 0$ for all t . Expand: $f(t) = \|x\|^2 - 2t \text{inner}(x, y) + t^2 \|y\|^2$. This quadratic is always non-negative, so discriminant ≤ 0 :

$$4\text{inner}(x, y)^2 - 4\|x\|^2 \|y\|^2 \leq 0 \quad (31)$$

Applications:

- Proves triangle inequality
- Bounds correlations: $|\text{Corr}(X, Y)| \leq 1$
- Controls Rademacher complexity (Week 3)

 Looking Ahead

We've seen that norm choice determines solution structure in finite dimensions.

But what about infinite-dimensional spaces? (Function spaces)

In finite dimensions: All norms equivalent (same topology) In infinite dimensions: **Norms NOT equivalent!** Different convergence notions

Brief preview: Functional analysis (next section) handles infinite dimensions, where norm choice becomes critical (L^2 vs L^1 vs L^∞ for functions give different spaces!)

Summary: Choosing the Right Norm

 Key Takeaway**When to use each norm:** **L^1 (Lasso):**

- Want sparse solutions (feature selection)
- High-dimensional data ($p \gg n$)
- Interpretability important
- Example: Gene selection, image compression

 L^2 (Ridge):

- Want all features to contribute
- Numerical stability important
- Natural for Euclidean geometry
- Example: Ridge regression, neural networks

 L^∞ (Max-norm):

- Want bounded maximum
- Robustness to outliers
- Minimax optimization
- Example: Robust statistics, game theory

Elastic Net ($L^1 + L^2$):

- Want sparsity AND stability
- Correlated features
- Best of both worlds

The principle: Norm is not just technicality—it's a design choice encoding prior knowledge about solutions!

Functional Analysis: Functions as Vectors

🧐 Student Question

From the class: “We’ve been working with finite-dimensional vectors in $\mathbb{R}^n$. Why do we need infinite-dimensional spaces like function spaces? Machine learning works with finite datasets—why care about infinite dimensions?”

This seems overly abstract for practical ML.

The Apparent Separation: Three Different Worlds

In mathematics, we work with different types of objects:

Domain	Objects	Operations
Linear Algebra	Vectors in $\mathbb{R}^n$	Dot product, matrix multiply
Analysis	Functions $f: \mathbb{R} \rightarrow \mathbb{R}$	Integration, differentiation
Discrete Math	Sequences (x_n)	Limits, sums

They seem completely different:

- Vectors: finite-dimensional, geometric
- Functions: continuous, infinite-dimensional
- Sequences: discrete, indexed by integers

Must we develop separate theories for each?

The Key Insight: They’re All Vector Spaces

💡 Aha Moment

The Unification: Hilbert Space Framework

Surprising discovery: Vectors, functions, and sequences all share the same structure!

They’re all **Hilbert spaces**—vector spaces with inner products that are complete.

Common structure:

- Addition: $x + y$
- Scalar multiplication: αx
- Inner product: $\text{inner}(x, y)$
- Norm: $\|x\| = \sqrt{\text{inner}(x, x)}$

Same theorems apply to all three!

Comparison

The same structure in three contexts:

Finite vectors ($\mathbb{R}^n$):

- Objects: $x = (x_1, \dots, x_n)$
- Inner product: $\text{inner}(x, y) = \sum_{i=1}^n x_i y_i$
- Norm: $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$

Functions ($L^2[0,1]$):

- Objects: $f : [0, 1] \rightarrow \mathbb{R}$ with $\int_0^1 f^2 < \infty$
- Inner product: $\text{inner}(f, g) = \int_0^1 f(x)g(x) \text{ dif } x$
- Norm: $\|f\|_2 = \sqrt{\int_0^1 f(x)^2 \text{ dif } x}$

Sequences (ℓ^2):

- Objects: $(x_1, x_2, x_3, \dots)$ with $\sum_{i=1}^{\infty} x_i^2 < \infty$
- Inner product: $\text{inner}(x, y) = \sum_{i=1}^{\infty} x_i y_i$
- Norm: $\|x\|_2 = \sqrt{\sum_{i=1}^{\infty} x_i^2}$

Key insight: Same formulas, just different types of “coordinates”!

Why This Matters for Machine Learning

Example: Kernel Methods as Infinite-Dimensional Learning

Finite-dimensional linear classifier:

- Data: $x \in \mathbb{R}^n$
- Hypothesis: $h(x) = \text{sign}(w^T x + b)$ where $w \in \mathbb{R}^n$
- Limited: Can only learn linear boundaries

Kernel trick (infinite-dimensional):

- Map to infinite dimensions: $\varphi : \mathbb{R}^n \rightarrow \mathcal{H}$ (Hilbert space)
- Hypothesis: $h(x) = \text{sign}(\text{inner}(w, \varphi(x)) + b)$ where $w \in \mathcal{H}$
- Never compute $\varphi(x)$ explicitly—only use kernel $k(x, y) = \text{inner}(\varphi(x), \varphi(y))$

Example: RBF kernel

$$k(x, y) = \exp\left(-\|x - y\|_{\frac{\cdot}{2\sigma^2}}^2\right) \quad (32)$$

This corresponds to infinite-dimensional feature map!

Why functional analysis matters: Ensures inner products, projections, and optimization work in infinite dimensions just like in $\mathbb{R}^n$

The Benefits of Unification

Before unification: Three separate theories

- Finite-dimensional linear algebra (matrix methods)
- Function spaces (calculus, PDEs)
- Sequence spaces (real analysis, convergence)

After unification: One framework handles all

- Cauchy-Schwarz inequality: works in all three
- Projection theorem: works in all three
- Convergence: same concepts in all three

Practical benefit: Tools from finite dimensions extend to infinite dimensions!

Key Takeaway

Why functional analysis for ML:

1. **Kernel methods:** Implicitly work in infinite dimensions (RKHS theory)
2. **Function approximation:** Neural networks approximate functions in L^2 space
3. **Regularization:** Penalties like $\int \|f''(x)\|^2$ control smoothness
4. **Consistency proofs:** Show learned functions converge to true function

The principle: “Functions are vectors” enables finite-dimensional intuition for infinite-dimensional problems

Keep it brief: This is supporting theory—main content is in concentration, convexity, and norms

Looking Ahead

Functional analysis provides the foundation for **kernel methods** and **reproducing kernel Hilbert spaces (RKHS)**.

Week 5 introduces kernel ridge regression:

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \quad (33)$$

where $\mathcal{H}$ is an infinite-dimensional Hilbert space!

The representer theorem shows: optimal f lies in span of $k(x_i, \cdot)$ —a finite-dimensional subspace, making the problem tractable.

Convexity: Why Gradient Descent Isn't Universal

🤔 Student Question

From the class: “Gradient descent is simple: just follow $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$ downhill. Can't we use it on any smooth function? Why does convexity matter?”

This seems reasonable—gradients point downhill, so following them should find the minimum, right?

The Natural Approach: Gradient Descent on Any Function

Gradient descent is the workhorse of machine learning. The algorithm is beautifully simple:

Algorithm:

1. Start at some θ_0
2. Repeat: $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$ (move opposite to gradient)
3. Stop when $\|\nabla f(\theta_t)\|$ is small

Intuition: The gradient points uphill, so moving opposite to it goes downhill. Keep going until you reach the bottom.

Why it seems universal:

- Works for linear regression: finds $(X^T X)^{-1} X^T y$
- Works for logistic regression: converges to optimal classifier
- Works for neural networks: trains deep learning models

So why do we care about convexity?

Where Gradient Descent Breaks: The Local Minimum Trap

Let's test gradient descent on a simple non-convex function.

Example: $f(x) = x^4 - 5x^3 + 6x^2$ on $\mathbb{R}$

This function has:

- Local minimum at $x \approx 0.5$ with $f(0.5) \approx 0.6875$
- Local minimum at $x \approx 2.5$ with $f(2.5) = -2.1875$ (global!)

Experiment:

- Start at $x_0 = 0.1$ (close to origin)
- Run gradient descent with $\alpha = 0.01$

Result:

- $x_1 = 0.1 - 0.01 \nabla f(0.1) \approx 0.113$
- $x_2 \approx 0.125$
- ...

- Converges to $x^* \approx 0.5$ (local minimum)
- $f(0.5) \approx 0.69$ (not the global minimum!)

The problem: Started near $x = 0.5$, got stuck there, never found the better minimum at $x = 2.5$ where $f(2.5) = -2.19$.

Gap: Missed the global minimum by $\Delta f = 0.69 - (-2.19) = 2.88$

⚠ Crisis

The Local Minima Catastrophe

Problem: Non-convex optimization with gradient descent

What happens:

- Gradient descent converges to local minimum x^*
- But there exists better point $x^\dagger$ with $f(x^\dagger) < f(x^*)$
- No way to escape: $\nabla f(x^*) = 0$ means algorithm stops!

Concrete failure:

- Function: $f(x) = x^4 - 5x^3 + 6x^2$
- Found: local min at $x = 0.5$ with $f = 0.69$
- Missed: global min at $x = 2.5$ with $f = -2.19$
- Gap: 2.88 (4× worse!)

For neural networks (non-convex):

- 1000 parameters $\rightarrow 2^{1000}$ potential local minima (exponentially many!)
- No guarantee of finding global optimum
- Initialization matters critically
- Learning rate tuning essential

Without convexity: No polynomial-time guarantee!

Quantifying the Problem: How Many Local Minima?

Simple non-convex functions: Can count explicitly

- $f(x) = x^4 - 5x^3 + 6x^2$: 2 local minima
- $f(x) = \sin(x)$: infinitely many local minima (at $x = -\frac{\pi}{2} + 2\pi k$)

Neural networks: Exponentially many

- Layer with n neurons: can have 2^n symmetries
- Network with L layers, n neurons each: $\approx 2^{nL}$ local minima
- Example: 10 layers $\times$ 100 neurons $\rightarrow 2^{1000} \approx 10^{300}$ local minima!

Impact:

- Random initialization $\rightarrow$ land in random local minimum
- No way to know if found global optimum
- Different runs give different solutions

- Need tricks: momentum, adaptive learning rates, careful initialization

Comparison

Convex vs Non-convex Optimization:

Property	Convex	Non-convex
Number of local minima	1 (global only)	Exponentially many
Gradient descent guarantee	Finds global min	Finds local min
Initialization sensitivity	Low	Very high
Learning rate tuning	Easier	Critical
Convergence rate (smooth)	$O(\frac{1}{t})$	None guaranteed
Convergence rate (strong)	$O(\exp(-t))$	N/A

Key difference: Convex has unique global minimum, non-convex has many local minima

Practical impact: Logistic regression (convex) reliably converges, neural networks (non-convex) need careful tuning

The Key Insight: Convexity Eliminates Local Minima

Aha Moment

The Resolution: Convexity Guarantees Global Optimum

Definition: Function f is convex if for all x, y and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \quad (34)$$

Geometric meaning: Function “curves upward”—any line segment between points on graph lies above the graph

Key property: Convex functions have NO local minima except the global minimum!

Why gradient descent works: Any point with $\nabla f(x) = 0$ is the global minimum

Definition: Convex Function

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **convex** if its domain is convex and for all x, y in the domain and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \quad (35)$$

Equivalently (first-order condition): For differentiable f :

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) \quad \text{for all } x, y \quad (36)$$

Equivalently (second-order condition): For twice-differentiable f :

$$\nabla^2 f(x) \preceq 0 \quad (\text{Hessian is positive semidefinite}) \quad (37)$$

Why the first-order condition matters: The tangent line at any point is a **global under-estimator**. This means:

- Following the gradient always decreases f
- When gradient is zero, we're at global minimum
- No local minima to get trapped in!

Examples: Convex vs Non-convex Functions

Comparison

Convex functions in ML:

- Linear regression: $f(\theta) = \|y - X\theta\|^2$ (quadratic, convex)
- Ridge regression: $f(\theta) = \|y - X\theta\|^2 + \lambda \|\theta\|^2$ (sum of convex, convex)
- Logistic regression: $f(\theta) = \sum_i \log(1 + \exp(-y_i \theta^T x_i))$ (convex)
- SVM: $f(w) = \|w\|^2 + C \sum_i \max(0, 1 - y_i w^T x_i)$ (convex)

Non-convex functions in ML:

- Neural networks: $f(W) = \sum_i \|y_i - \sigma(W_L \dots \sigma(W_1 x_i))\|^2$ (composition of affine + non-linear)
- Matrix factorization: $f(U, V) = \|R - UV^T\|^2$ (product of variables)
- K-means: $f(\mu) = \sum_i \min_k \|x_i - \mu_k\|^2$ (min over discrete choices)

Key observation: Traditional ML (regression, SVM, logistic) uses convex formulations. Deep learning sacrifices convexity for expressiveness.

Why Convexity Matters: Convergence Guarantees

Theorem: Optimality for Convex Functions

Let f be differentiable and convex. Then:

1. Any local minimum is a global minimum
2. If $\nabla f(x^*) = 0$, then x^* is a global minimum
3. The set of global minima is convex

Proof (part 2): Suppose $\nabla f(x^*) = 0$. By first-order condition:

$$f(y) \geq f(x^*) + \nabla f(x^*)^T (y - x^*) = f(x^*) + 0 = f(x^*) \quad (38)$$

So $f(y) \geq f(x^*)$ for all y , meaning x^* is global minimum! ■

Practical impact: For convex functions, gradient descent with appropriate step size always finds global minimum.

Strong Convexity: Exponential Convergence

Convexity alone gives $O(\frac{1}{t})$ convergence. But with strong convexity, we get exponential convergence!

Definition: Strong Convexity

Function f is **μ -strongly convex** if for all x, y :

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2} \|y - x\|^2 \quad (39)$$

Equivalently: $\nabla^2 f(x) \succeq \mu I$ (Hessian bounded below by μI)

Intuition: Strong convexity means function curves upward at least as fast as $\frac{\mu}{2} \|x\|^2$. This prevents flat regions.

Comparison

Convergence rates for gradient descent:

Function Class	Convergence Rate	Iterations for ε error
Non-convex	None guaranteed	∞ (no guarantee)
Convex	$O(\frac{1}{t})$	$O(\frac{1}{\varepsilon})$
Strongly convex	$O(\exp(-\mu t))$	$O(\log(\frac{1}{\varepsilon}))$

Example: For $\varepsilon = 0.001$ accuracy:

- Convex: ≈ 1000 iterations
- Strongly convex ($\mu=0.1$): ≈ 70 iterations
- **14× speedup from strong convexity!**

Why it matters: Ridge regression (strongly convex with $\mu = \lambda$) converges exponentially fast

Checking Convexity: The Hessian Test

Practical question: How do we check if a function is convex?

Answer: Compute the Hessian and check if it's positive semidefinite.

Example: Ridge Regression

Ridge regression minimizes:

$$f(\theta) = \|y - X\theta\|^2 + \lambda \|\theta\|^2 \quad (40)$$

Gradient:

$$\nabla f(\theta) = -2X^T(y - X\theta) + 2\lambda\theta = 2(X^T X + \lambda I)\theta - 2X^T y \quad (41)$$

Hessian:

$$\nabla^2 f(\theta) = 2(X^T X + \lambda I) \quad (42)$$

Is it positive semidefinite? For any $v \in \mathbb{R}^n$:

$$v^T \nabla^2 f(\theta) v = 2v^T (X^T X + \lambda I) v = 2(\|Xv\|^2 + \lambda \|v\|^2) \geq 2\lambda \|v\|^2 \geq 0 \quad (43)$$

Conclusion: Ridge regression is convex! In fact, it's λ -strongly convex.

Impact: Gradient descent on ridge regression converges exponentially fast with rate $\exp(-\mu t)$ where $\mu = \lambda$.

Example: Logistic Regression

Logistic regression minimizes:

$$f(\theta) = \sum_{i=1}^n \log(1 + \exp(-y_i \theta^T x_i)) \quad (44)$$

Hessian:

$$\nabla^2 f(\theta) = \sum_{i=1}^n \frac{\exp(-y_i \theta^T x_i)}{(1 + \exp(-y_i \theta^T x_i))^2} x_i x_i^T \quad (45)$$

Each term is of form $p_i x_i x_i^T$ where $p_i \in (0, 1)$, so:

$$v^T \nabla^2 f(\theta) v = \sum_{i=1}^n p_i (v^T x_i)^2 \geq 0 \quad (46)$$

Conclusion: Logistic regression is convex (positive semidefinite Hessian)!

Impact: Gradient descent reliably finds global optimum, no matter the initialization.

The Cost of Non-convexity: Neural Networks

Example: Why Training Neural Networks Is Hard

Single hidden layer network with ReLU activation:

$$f(\theta) = \sum_{i=1}^n \left(y_i - \sum_{j=1}^m w_j \max(0, v_j^T x_i) \right)^2 \quad (47)$$

where $\theta = (w_1, \dots, w_m, v_1, \dots, v_m)$ are the weights.

Problem 1: Product of variables The term $w_j \max(0, v_j^T x_i)$ is a product $w_j \cdot \text{activation}(v_j)$.

Products of optimization variables $\rightarrow$ non-convex!

Problem 2: Symmetry Swapping neurons j and k gives same function:

$$(w_j, v_j, w_k, v_k) \rightarrow (w_k, v_k, w_j, v_j) \quad (48)$$

With m neurons: $m!$ equivalent solutions (all local minima)

For $m = 10$: $10! = 3,628,800$ local minima just from symmetry!

Problem 3: Hessian can be indefinite Second-order term can have negative eigenvalues $\rightarrow$ saddle points

Consequence:

- Initialization matters: different starts $\rightarrow$ different local minima
- Learning rate critical: too large diverges, too small stuck
- Need tricks: momentum, adaptive rates, batch normalization
- No guarantee of global optimum

Despite non-convexity, neural networks work because:

- Many local minima are “good enough” (similar performance)
- Overparameterization helps (more parameters than data)
- Stochastic gradient descent escapes saddle points

Convex Formulations in Machine Learning

Many ML algorithms are designed to be convex:

Algorithm	Formulation	Why Convex
Linear Regression	$\min \ y - X\theta\ ^2$	Quadratic in θ
Ridge Regression	$\min \ y - X\theta\ ^2 + \lambda \ \theta\ ^2$	Sum of convex
Lasso Regression	$\min \ y - X\theta\ ^2 + \lambda \ \theta\ _1$	Quadratic + L^1 norm
Logistic Regression	$\min \sum \log(1 + \exp(-y_i \theta^T x_i))$	Log-exp is convex
SVM	$\min \ w\ ^2 + C \sum \max(0, 1 - y_i w^T x_i)$	Quadratic + hinge loss

Design principle: When possible, formulate ML problems as convex optimization to get:

- Guaranteed convergence to global optimum
- Algorithm-independent solution (any solver works)

- Polynomial-time solvability
- Interpretable solutions

Looking Ahead

We've seen that convexity guarantees gradient descent finds the global minimum.

But how fast does it converge?

For smooth convex functions with Lipschitz gradient: $O(\frac{1}{t})$ iterations

For strongly convex functions with condition number κ : $O(\exp(-(\frac{\mu}{L})t))$ iterations

Week 10 (Online Learning) analyzes convergence rates in detail and introduces accelerated methods (Nesterov, AdaGrad, Adam) that improve these rates.

Key Takeaways

Key Takeaway

Why convexity matters for machine learning:

1. **Global optimality:** Any local minimum is global (no traps!)
2. **Algorithmic guarantee:** Gradient descent with proper step size converges
3. **Initialization insensitive:** Starting point doesn't matter (for convex)
4. **Convergence rates:** $O(\frac{1}{t})$ for convex, $O(\exp(-t))$ for strongly convex
5. **Design principle:** Formulate ML problems as convex when possible

The trade-off:

- Convex models (linear, logistic, SVM): Less expressive, guaranteed convergence
- Non-convex models (neural networks): More expressive, harder optimization

Modern practice: Deep learning accepts non-convexity for expressiveness, uses tricks (momentum, adaptive learning rates) to cope with optimization challenges.

Complexity Theory: Why $n^2 \neq 2^n$

🧐 Student Question

From the class: “I understand that algorithms have different running times—some are $O(n^2)$, some are $O(2^n)$. But both grow as n increases, right? Why does the difference matter so much?”

This seems like a quantitative difference, not a qualitative one.

The Natural Intuition: All Growth Functions Similar

When you first learn about computational complexity, the focus is on asymptotic behavior:

- $O(n)$: linear time
- $O(n^2)$: quadratic time
- $O(2^n)$: exponential time

All grow to infinity as $n \rightarrow \infty$, so they all get slow eventually, right?

Where This Breaks: Concrete Numbers

Let's compute actual running times for $n = 100$:

📊 Comparison

Running time comparison ($n = 100$):

Assume: 1 billion operations per second (modern CPU)

Complexity	Operations	Time	Practical?
$O(n)$	100	0.0001 ms	✓ Instant
$O(n \log n)$	664	0.0007 ms	✓ Instant
$O(n^2)$	10,000	0.01 ms	✓ Very fast
$O(n^3)$	1,000,000	1 ms	✓ Fast
$O(2^n)$	$\approx 10^{30}$	$\approx 10^{13}$ years	✗ Impossible!

The gap: 1 millisecond vs 10 trillion years!

This is not a quantitative difference—it's a **qualitative barrier** between tractable and intractable.

⚠ Crisis**The Exponential Catastrophe**

Problem: Many ML problems are NP-hard (exponential worst-case)

Examples of NP-hard ML problems:

- Optimal decision tree learning: $O(2^n)$ where n = features
- Best subset selection in regression: $O(2^p)$ where p = number of features
- Exact inference in graphical models: $O(2^n)$ where n = variables
- Neural network training with discrete weights: NP-hard

Concrete impact ($n = 1000$ features):

- Polynomial (n^2): 10^6 operations $\approx$ 1 millisecond ✓
- Exponential (2^n): 2^{1000} operations $\approx 10^{280}$ years ✗

Without polynomial-time algorithms, these problems are intractable!

Reality check:

- Age of universe: $\approx 10^{10}$ years
- Atoms in universe: $\approx 10^{80}$
- With 2^{1000} operations: impossible even with every atom as a processor!

Why Hardware Doesn't Save Us

You might think: “Just wait for faster computers!”

Moore's Law: Computing power doubles every 2 years

What does this buy us?

Comparison

Impact of $1000\times$ speedup:

Today: Solve problem of size n in 1 second

After $1000\times$ speedup (20 years of Moore's Law):

Complexity	New Size	Improvement
$O(n)$	$1000n$	$1000\times$ bigger problems
$O(n^2)$	$31.6n$	$32\times$ bigger problems
$O(2^n)$	$n + 10$	Only 10 more features!

The exponential barrier:

- Polynomial: Hardware speedup helps significantly
- Exponential: Hardware speedup helps negligibly

Why: $2^{(n+10)} = 2^n \cdot 2^{10} = 1024 \cdot 2^n \approx 1000 \cdot 2^n$

$1000\times$ faster computer $\rightarrow$ solve problem only 10 units larger!

The Key Insight: P vs NP

Aha Moment

The Resolution: Polynomial vs Exponential Tractability

Complexity classes:

- **P:** Problems solvable in polynomial time $O(n^k)$
 - Linear regression, sorting, shortest path
 - Tractable even for large inputs
- **NP:** Problems verifiable in polynomial time
 - Includes P, but also harder problems
 - Optimal decision tree, graph coloring, SAT
 - Unknown if solvable in polynomial time
- **NP-hard:** At least as hard as hardest NP problems
 - Many ML problems fall here!

$P \neq NP$ conjecture: Most experts believe exponential time is unavoidable for NP-hard problems

Practical impact: Need approximation algorithms or heuristics for NP-hard problems

ML Examples: Tractable vs Intractable

Comparison

Tractable (Polynomial-time):

- Linear regression: $O(np^2)$ where n = samples, p = features
- Gradient descent (convex): $O(npd/\epsilon)$ where d = dimension
- PCA: $O(np^2)$ via SVD
- K-means (heuristic): $O(nkt)$ where k = clusters, t = iterations

Intractable (NP-hard):

- Optimal decision tree: $O(2^n)$ where n = features
- Best subset selection: $O(2^p)$ where p = features
- Exact inference in Bayes nets: $O(2^n)$ where n = variables
- Training neural net with discrete weights: NP-hard

Solution for intractable problems: Use approximations!

- Greedy algorithms (CART for decision trees)
- Regularization (Lasso instead of best subset)
- Variational inference (approximate Bayesian inference)
- Continuous relaxation (continuous weights for neural nets)

Decision Trees: Optimal vs Practical

Example: Why We Use CART Instead of Optimal Trees

Optimal decision tree problem:

- Input: Dataset with n samples, d features
- Goal: Find smallest tree with perfect accuracy
- Complexity: $O(2^d)$ (NP-hard!)

For $d = 100$ features:

- Optimal: $2^{100} \approx 10^{30}$ operations (intractable)

CART algorithm (greedy heuristic):

- At each node: try all features, all splits
- Complexity: $O(nd^2 \log n)$ (polynomial!)
- For $n = 10,000$, $d = 100$: $\approx 10^8$ operations (1 second)

Trade-off:

- Optimal: Guaranteed smallest tree, but intractable
- CART: Near-optimal tree, tractable in practice

Why CART works: Greedy locally optimal choices often give globally good (not optimal) solutions

Quantum Computing: Does It Help?

You might wonder: “Can quantum computers solve NP-hard problems?”

Short answer: Probably not!

- Grover’s algorithm: $O(\sqrt{2^n})$ for search (quadratic speedup)
- Still exponential: $\sqrt{2^{1000}} = 2^{500}$ (still intractable!)
- $P \neq NP$ likely holds even for quantum computers

Why quantum doesn’t save us:

- $\sqrt{2^n}$ is still exponential, just smaller exponent
- For $n = 1000$: $2^{500} \approx 10^{150}$ (still astronomically large)

Key Takeaway

The tractability divide:

Polynomial time: $n^2 = 10,000$ vs $2^n = 10^{30}$

- Factor of 10^{26} difference!
- Polynomial: tractable
- Exponential: intractable

Why it matters for ML:

- Many problems NP-hard in worst case
- Need polynomial approximations
- Design principle: Find tractable relaxations

The principle: Algorithm design often trades optimality for tractability

Connection to Learning Theory

Many learning problems have polynomial sample complexity but exponential computational complexity:

- **Sample complexity:** $O(d/\epsilon^2)$ samples (polynomial in dimension d)
- **Computational complexity:** Finding optimal hypothesis may be NP-hard

Examples:

- Learning decision trees: Polynomial samples, exponential computation
- Learning neural networks: Polynomial samples, non-convex optimization

The gap: Statistical learnability $\neq$ computational learnability

 Looking Ahead

Week 2 shows that hypothesis classes with finite VC dimension are PAC-learnable.

But PAC learning only guarantees polynomial sample complexity, not polynomial time!

Some classes are statistically learnable (polynomial samples) but computationally hard (no efficient algorithm known).

This gap between information-theoretic and computational complexity is an active research area.

Week 1 Synthesis: The Mathematical Toolkit

We've built the foundation for learning theory. Here's how the pieces fit together:

The Four Key Tools

Tool	Key Insight	Impact on Learning
Concentration	Hoeffding: exponential decay ($e^{-m\varepsilon^2}$)	270× fewer samples than Markov bound
Norms	Choice encodes inductive bias (L^1 sparse, L^2 dense)	Lasso: 20,000 features $\rightarrow$ 50 (400× sparsity)
Convexity	No local minima (1 global vs 2^{1000} local)	Logistic regression reliable, neural nets need tricks
Complexity	Polynomial tractable, exponential intractable	$n^2=10,000$ (1 ms) vs $2^{100}=10^{30}$ (impossible)

Quantitative Impact Summary

The cognitive journey reveals concrete numerical gaps:

Concept	Naive Approach	Proper Tool	Improvement
Sample complexity	40,000 (Markov)	148 (Hoeffding)	270×
Feature selection	20,000 features	50 (Lasso)	400×
Optimization	2^{1000} local minima	1 global (convex)	Exponential
Decision tree	2^{100} ops (optimal)	10^8 ops (CART)	10^{22} ×

Key insight: Mathematical “details” aren’t academic—they determine whether algorithms work in practice.

Cross-Cutting Themes

Theme 1: Exponential vs Polynomial

- Concentration: $e^{-m\varepsilon^2}$ vs $\frac{1}{m\varepsilon^2} \rightarrow 270\times$ sample gap
- Convexity: 1 minimum vs 2^n minima $\rightarrow$ reliable vs unreliable
- Complexity: n^2 vs $2^n \rightarrow 1$ ms vs age of universe

Pattern: Exponential differences dominate practical feasibility.

Theme 2: Structure Enables Tractability

- Boundedness + independence $\rightarrow$ Hoeffding (exponential concentration)
- Convexity $\rightarrow$ gradient descent works (polynomial iterations)

- Polynomial time $\rightarrow$ tractable algorithms

Pattern: Additional structure (constraints, convexity) makes problems solvable.

Theme 3: Design Choices Matter

- Norm choice: L^1 vs $L^2 \rightarrow$ sparse vs dense solutions
- Convex formulation: logistic regression vs neural networks
- Approximation: CART vs optimal decision tree

Pattern: Algorithm design trades optimality for tractability.

Looking Ahead to Week 2

This week established tools. Week 2 combines them into PAC learning:

The PAC learning question: Given hypothesis class $\mathcal{H}$, how many samples m guarantee:

$$\text{Prob}\left[R(\hat{h}) \leq \min_{h \in \mathcal{H}} R(h) + \varepsilon\right] \geq 1 - \delta \quad (49)$$

Answering requires:

1. **Hoeffding's inequality** (Week 1): Bounds single hypothesis deviation
2. **Union bound** (Week 2): Extends to all hypotheses in $\mathcal{H}$
3. **VC dimension** (Week 2): Measures effective complexity
4. **Sauer's lemma** (Week 2): Converts VC dim to sample complexity

Preview: For class with VC dimension d :

$$m = O\left(\frac{d + \log\left(\frac{1}{\delta}\right)}{\varepsilon^2}\right) \quad (50)$$

This formula combines probability (Hoeffding), geometry (VC dimension), and complexity (polynomial in $d, \frac{1}{\varepsilon}$).

Common Misconceptions Resolved

Wrong Belief	Correct Understanding
"Mean is enough for bounds"	Need variance (Chebyshev) or boundedness (Hoeffding) for tight bounds
"All norms are basically the same"	Norm choice encodes inductive bias: L^1 sparse, L^2 dense, L^∞ bounded-max
"Gradient descent works on any function"	Non-convex: 2^{1000} local minima. Convex: unique global minimum
"Both n^2 and 2^n get slow eventually"	$n^2=10,000$ (tractable) vs $2^n=10^{30}$ (intractable) —qualitative difference
"More data always helps"	Diminishing returns after $O(d/\varepsilon^2)$ samples— data efficiency matters

Worked Example: Credit Scoring Revisited

Let's apply all four tools to credit scoring with 100 features:

Problem: Predict loan default from 100 features (income, debt, credit history, ...)

Probability (Hoeffding):

- Want: $\text{Prob}(|\text{error} - \text{training error}| > 0.05) \leq 0.05$
- Need: $m \geq \frac{\log(\frac{2}{0.05})}{2 \cdot 0.0025} \approx 148$ samples
- With Markov: Would need 40,000 samples (270× worse!)

Norms (regularization choice):

- L^2 (Ridge): All 100 features contribute, stable predictions
- L^1 (Lasso): Select 10 key features, interpretable model
- Choice depends on goal: prediction vs interpretation

Convexity (algorithm):

- Logistic regression: Convex, gradient descent finds global optimum reliably
- Neural network: Non-convex, needs careful initialization and tuning
- Trade-off: expressiveness vs optimization guarantees

Complexity (algorithm design):

- Optimal feature selection: 2^{100} subsets (intractable)
- Lasso: Convex relaxation, polynomial time
- Result: Near-optimal in practice, tractable

Integration: All four tools combine to make credit scoring practical!



Confidence Check

Before moving to Week 2, ensure you can:

- ☐ Explain why Hoeffding gives 270× better sample bound than Markov
- ☐ Choose between L^1 and L^2 regularization for a given problem
- ☐ Explain why logistic regression is easier to optimize than neural networks
- ☐ Distinguish polynomial (tractable) from exponential (intractable) complexity

If any checkbox is unclear, review the relevant section!

Final Takeaway

Key Takeaway**Why these mathematical tools matter:**

Not just theory: Each tool provides orders-of-magnitude practical improvements

- Hoeffding: $270\times$ fewer samples
- Lasso: $400\times$ sparsity
- Convexity: Reliable vs unreliable optimization
- Polynomial time: Tractable vs intractable

Design principle: Choose formulations that leverage mathematical structure

- Use Hoeffding (not Markov) for concentration bounds
- Choose norm to match problem structure (sparse vs dense)
- Prefer convex formulations when possible
- Use polynomial-time approximations for NP-hard problems

The paradigm: Mathematics isn't overhead—it's the difference between algorithms that work and those that don't.